NPLify · P0 Technical Baseline

Configuration Schema Specification

New XP Technologies Limited

22 October 2026 · Confidential

How to read this pack

Four map pages are now part of the pack as documents of their own, and are the place to start: the Top-Down Map (from the business, through the data model and the deal states, to the scenarios and the reading order), the Deal Flow on one page (Quote, Collect, Convert, Settle, with the roles and the money path), the Workflow Tree (8 classes, 27 sub-classes, 92 procedures, each naming who initiates, who approves, the codes it is written under and the records it touches) and the Workflow Tree on one page. Read them in that order, then this document. Every worked transaction (T), acceptance vector (L, B), walkthrough (W, X), flow (F) and procedure (C, A, D) has one identifier, listed in the Reading Companion's identifier register; the Test Strategy & Coverage document says which test proves each procedure.

1 · How configuration is organised

Four ideas carry the whole model. **A client has projects.** The client is the commercial relationship; each project is one arrangement with it, with its own settings, currency pairs, senders, receiver groups and partner configurations. **Parties are partner-independent.** Senders, receiver groups and receiving entities belong to the project, so the same Sender A → Receiver Y flow can run through Ali today and Jeton tomorrow. **How a partner is used is a block.** Fees and tiers, vehicles and their collection endpoints, the narrative and the rail that pay out, the destination approvals and the rebate rule all hang off one PARTNER_CONFIG, and a deal leg stamps the block's rows it was priced and routed on. **The partner's clock is the partner's.** Cutoff, time zone, holidays and overrides belong to the global PARTNER and apply wherever a pair says it relies on the banking channel.

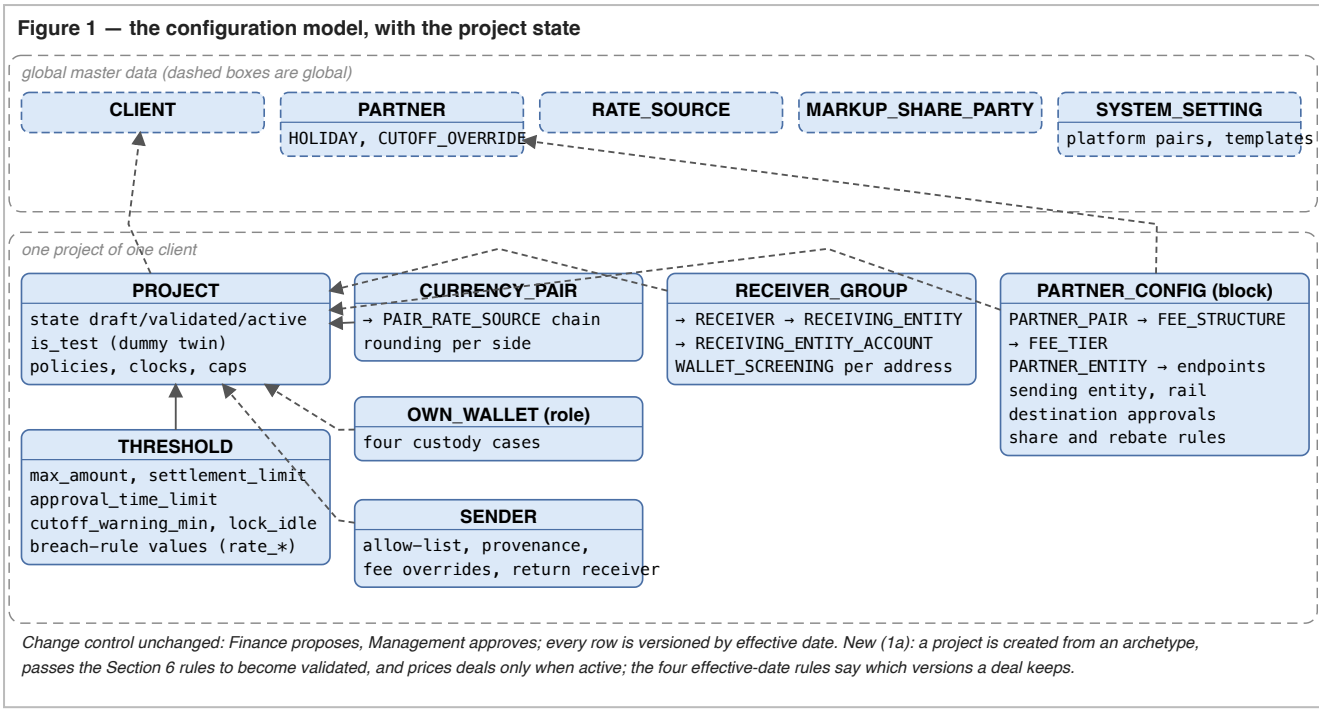


Figure 1 — The configuration model, with the project state (PROJECT.state, PROJECT.is_test) and the THRESHOLD kinds. Dashed boxes are global; everything inside the shaded area belongs to one project of one client.

1a · Creating a project, project states and effective-date rules

Creating a project from an archetype

A new project is not set up field by field. It is created from one of the six project archetypes of the Client Operating Procedures (Part C): the archetype pre-fills the project's policies, its currency pairs with their rate-source chains, the fee structure and tiers of each partner pair, the message templates, the thresholds and the partner configuration block. Finance then changes only what differs for the client (names, parties, accounts, the fee percentages agreed) and proposes the project; Management approves it as one change. An existing project of the same archetype may be used as the source instead of the archetype itself (cloning), with the same rules. The guided setup screens that do this are built in milestone M0 of Phase 1.

Project states: draft, validated, active

A project carries a state in addition to its status. **draft**: the project is being set up; rows may be added and changed freely under the normal proposal and approval rules; no deal can be priced on it. **validated**: every cross-entity rule of Section 6 passes for the project and for each of its partner configuration blocks; the validation report lists each rule with its result and the rows concerned, and is kept with the project. **active**: Management has activated the validated project; deals may be priced and partner configurations used. A project returns to draft when a change proposal is approved that makes a rule fail, and must be validated again before it is used. A partner configuration block added later follows the same path on its own: it cannot be used on a deal until it passes the rules of Section 6. The validation report screen is built in M0; the rules themselves already exist (Section 6) and are evaluated on save.

Effective-date rules

Configuration is versioned by effective date (Change control, below). The four rules below say what a new version does to deals that already exist. They are stated in the same words in section 2 of the Lifecycle & Approvals document.

Rule	What happens
1. New versions apply to new pricing only	An approved configuration change takes effect for deals priced on or after its effective date. A deal priced before that date is not re-priced.
2. A priced deal keeps its versions	The fee structure version, the tier, the rate basis, the rounding options, the partner configuration and the policy versions stamped at pricing stay with the deal to the end of its life, including through settlement, return and loss handling.
3. A re-quote uses the configuration of the new pricing time	An expired or requoted leg is priced afresh on the versions in force when the new quote is made (a new deal group); the original leg keeps its stamp.
4. A collect-first deal uses the configuration in force when it is priced	A collect-first leg is priced at conversion, so it uses the versions effective at that time, not at the time the money arrived, unless a documented client agreement names a different rule for that project.

Worked scenarios (the fee rises from 2.0 % to 2.2 % effective 1 November; proposed 28 October, approved 29 October). A: quoted 30 October, collected 31 October, converted 2 November, pays 2.0 % (rule 2). B: quoted 2 November, pays 2.2 % (rule 1). C: quoted 30 October, expired, re-quoted 2 November, pays 2.2 % (rule 3). D: collected 31 October with no prior quote, priced at conversion on 2 November, pays 2.2 % (rule 4). These four scenarios are demonstrated in Phase 1 (M4) for fees, tiers, rounding options and partner settings, not only the headline percentage.

Change control. Configuration is versioned: thresholds, fee structures, tiers, templates, system settings and policies are new rows with an effective date, never edits. Finance proposes, Management approves (approver ≠ proposer, from the ROLE_RIGHT rows), and every change is audited. Deals keep the versions they were priced on. Three rows are not approvals but audited records: a partner cutoff override (Finance, with the partner's agreement), a wallet screening (Operations) and a destination approval's status as reported by the partner.

2 · Global master data and system settings

CLIENT · Global

The commercial relationship: the counterparty NPL contracts with and reports on (Betfair, Evo, 66 Group, GDC …). A client has one or more PROJECTs — BF, BF Sub and BF Withdrawal are three arrangements with one client — each with its own fee structures, partners and policies. Client-level figures (balances at partners, entitlement, custody, earnings, rebates) are roll-ups over the client's projects (CLIENT_POSITION), never stored; an optional exposure_ceiling caps the sum of what rests at partners across the client's projects.

Field	Type	Required	Allowed values · default · meaning
`name`	string	yes	Betfair, Evo, 66 Group, GDC, NPL-GR
`category`	string	no	NPL's own grouping (Project Context tab: BF, Evo, Game Reseller …)
`legal_entity_ref`	string	no	contracting entity, external store
`kyc_ref`	string	no	external store
`relationship_owner`	int	no	USER
`exposure_ceiling`	decimal	no	optional cap on Σ balance resting at partners across the client's projects, stated in the reporting currency and compared through REPORTING_VALUE at the latest market rate
`status`	string	yes	

- The commercial relationship NPL contracts with and reports on. A client has one or more projects: BF, BF Sub and BF Withdrawal are three arrangements with one client.

- exposure_ceiling, when set, caps the sum of what rests at partners across the client's projects; it is stated in the reporting currency and compared through REPORTING_VALUE.
- Client-level figures (CLIENT_POSITION) are roll-ups over the client's projects, never stored.

PARTNER · Global

A global counterparty (Ali, Jeton, PT): remittance companies, fintechs, EMIs, neobanks or crypto exchanges. How a partner is used in a project lives in PARTNER_CONFIG. disclosure_style records whether the partner's markup is visible. carries the partner's daily bank cutoff, like its rate expiry and holiday calendar — a property of the partner, whichever project uses it. supported_methods says which collection and settlement methods the partner offers; cash and local deposits have no standing endpoint, so they are declared here, while networks are read from the vehicle's wallet endpoints.

Field	Type	Required	Allowed values · default · meaning
`name`	string	yes	Ali, Jeton, PT
`disclosure_style`	string	yes	fee_in_rate / fee_on_market — whether the partner's fee sits inside its rate or is stated on top of market
`rate_expiry_time`	string	no	
`holiday_calendar`	string	no	name of the source calendar (e.g. ID banking holidays); the dates are HOLIDAY rows
`supported_methods`	string	yes	crypto / bank / cash / local_deposit — the methods this partner offers (Ali: all four; Jeton and Aquanow: crypto, bank). Networks are implied by its wallet endpoints
`status`	string	yes	
`bank_cutoff_time`	time	no	nullable: the partner's standing daily bank cutoff for pairs that rely on the banking channel (Ali and other Asian partners, 14:30); empty for Jeton and Aquanow, which bind nothing. Governs locking the rate for same-day conversion and initiating an outgoing bank settlement; per-day changes in PARTNER_CUTOFF_OVERRIDE
`cutoff_timezone`	string	no	IANA zone, e.g. Asia/Hong_Kong

- A partner is set up once and reused by any project through a PARTNER_CONFIG.
- disclosure_style says where the partner's fee sits: fee_in_rate (inside its rate — Ali) or fee_on_market (stated on top of market — Jeton).
- supported_methods lists the collection and settlement methods the partner offers (crypto / bank / cash / local_deposit); networks are implied by its vehicles' wallet endpoints.
- bank_cutoff_time with cutoff_timezone is the partner's standing daily bank cutoff; empty for a partner whose channel binds nothing (Jeton, Aquanow). It is required if any of the partner's pairs has needs_bank_lock.
- holiday_calendar names the source calendar; the dates are HOLIDAY rows.

HOLIDAY · Per partner

A non-business day of a partner's banking channel: one row per date (Indonesian bank holidays for Ali's PT vehicles, UK holidays for Jeton). The dates behind every 'next business day' in the model — a leg's conversion_due_date and a settlement's execution_due_date skip weekends in the partner's cutoff_timezone and the partner's HOLIDAY rows. scope says what is closed: the bank channel only (crypto pairs unaffected) or the partner altogether. PARTNER.holiday_calendar stays as the name of the source calendar; this table holds its dates.

Field	Type	Required	Allowed values · default · meaning
`partner_id`	int	yes	reference
`date`	date	yes	
`name`	string	no	Idul Fitri, Boxing Day

Field	Type	Required	Allowed values · default · meaning
scope	string	yes	bank_channel / partner · default **bank_channel**
source	string	no	partner notice / public calendar

- One row per non-business day of the partner's banking channel; conversion_due_date and execution_due_date skip these dates and weekends in the partner's cutoff_timezone.
- scope bank_channel leaves crypto pairs unaffected; scope partner closes the partner altogether.

PARTNER_CUTOFF_OVERRIDE · Per partner

A change to a partner's bank cutoff for one day or for a recurring weekday: an extension of 30–60 minutes that the partner agreed for an urgent request, or an early close on a Friday or the day before a holiday. The effective cutoff for a day is the one-day override if any, else the weekday rule, else PARTNER.bank_cutoff_time. Finance records it with the partner's agreement.

Field	Type	Required	Allowed values · default · meaning
partner_id	int	yes	reference
applies_on	date	no	one day; null for a weekday rule
weekday	string	no	recurring rule (Friday); null for a one-day override
cutoff_time	time	yes	the cutoff that applies instead
kind	string	yes	extension / early_close
reason	string	yes	urgent request / pre-holiday / partner notice
partner_agreement_ref	string	no	the partner agreed (extensions)
recorded_by	int	yes	Finance

- Exactly one of applies_on (a one-day override) and weekday (a recurring rule, e.g. Friday) is set.
- The effective cutoff for a day is the one-day override if any, else the weekday rule, else PARTNER.bank_cutoff_time.
- kind extension needs partner_agreement_ref; Finance records it, no approval (an audited step).

RATE_SOURCE · Global

A market data source. Cross-source slippage is a query over MARKET_RATE across sources for the same pair and window.

Field	Type	Required	Allowed values · default · meaning
code	string	yes	KRAKEN / CMC / OANDA / XE
staleness_threshold_s	int	yes	

- A snapshot older than staleness_threshold_s is not used for pricing; the next source in the pair's chain is tried; with every source down a manual rate needs Finance approval with evidence.

MARKUP_SHARE_PARTY · Global

A party who shares NPL's markup: an external markup-share party who brought in a client, or a customer such as Raeen who provides sub-accounts. Paid either a share of NPL's earnings or a fixed % per transaction; several markup-share parties may share one deal. may also be a client party (Raeen) who provides sub-accounts and shares NPL's net markup — MARKUP_SHARE_RULE.party_kind says which.

Field	Type	Required	Allowed values · default · meaning
name	string	yes	
status	string	yes	

- A party who shares NPL's markup: an external party who brought in a client, or a customer who provides sub-accounts. How it is paid on a project is MARKUP_SHARE_RULE.

SYSTEM_SETTING · Global, versioned

The system-settings area, kept simple: this key/value table (reporting_currency, reporting_rate_max_age_min, operating_timezone, and later keys), the ROLE table for user roles, and the platform-scope CURRENCY_PAIRs that carry the conversion rates for reporting through the ordinary PAIR_RATE_SOURCE chain. Every row is versioned by effective_from and approved by Management. Nothing here changes a price, a posting or a settlement.

Field	Type	Required	Allowed values · default · meaning
`key`	string	yes	reporting_currency / reporting_rate_max_age_min / operating_timezone / … (the reporting rate source is the platform pair's PAIR_RATE_SOURCE, not a key)
`value`	string	yes	
`effective_from`	datetime	yes	
`approved_by`	int	yes	Management

- Keys today: reporting_currency, reporting_rate_max_age_min, operating_timezone. A new value is a new row with a later effective_from, approved by Management.
- The reporting rate source is not a key: it is the PAIR_RATE_SOURCE chain of the platform-scope CURRENCY_PAIR (currency → reporting currency).
- Nothing here changes a price, a posting or a settlement.

MESSAGE_TEMPLATE · Global default, or per project

A versioned template for a message the system sends or renders: the quote package to a sender, the payment instructions, a confirmation request to a receiver, a settlement advice. Global, or owned by a project that needs its own wording. A template names the fields it prints; a template of purpose quote may print only sender-facing figures (sender rate, customer-facing fee and rate, amount out, validity) — never partner rates, margins or costs — which is how 's 'Operations never receives derived economics' is enforced on outgoing messages. Changing a template is a configuration change approved by Management.

Field	Type	Required	Allowed values · default · meaning
`project_id`	int	no	null = global default; a project row overrides it
`code`	string	yes	quote / payment_instructions / confirmation_request / settlement_advice / cash_token
`version`	int	yes	
`channel`	string	yes	email / chat / portal / pdf
`language`	string	yes	
`body_ref`	string	yes	external content store
`printed_fields`	string	yes	the model fields the template may print; sender-facing only for quote
`effective_from`	datetime	yes	
`approved_by`	int	yes	Management
`status`	string	yes	

- project_id null is the global default; a project row with the same code overrides it.
- printed_fields lists the model fields the template may print. A template of code quote may print only sender-facing figures (sender rate, customer-facing fee and rate, amount out, validity) — never a partner rate, a margin or a cost; the save is refused otherwise.
- A new version is a new row approved by Management; the version a package was rendered with is recorded on QUOTE_PACKAGE.

BANK_DETAILS_TEMPLATE · Platform, per rail

Configuration: what a bank account on a given rail must carry. One row per rail × field, in display order: SWIFT (account number or IBAN, BIC, bank address, intermediary bank), SEPA (IBAN, BIC), UK Faster Payments (sort code, account number), a local rail (bank code, account number). The template drives the entry form, the validation on save (IBAN check digits, BIC length, sort-code pattern) and the card that shows the details in full; wallets are not templated (network and address stay on the account row). Versioned by effective date like every configuration table; Finance proposes, Management approves.

Field	Type	Required	Allowed values · default · meaning
`rail`	string	yes	SWIFT / SEPA / UK_FPS / local:<country> ...; one template per rail
`position`	int	yes	display order within the rail
`field_key`	string	yes	iban, bic, account_number, sort_code, bank_code, bank_address, intermediary_bic ...
`label`	string	yes	as shown on the form and the card
`required`	bool	yes	· default **true**
`format_rule`	string	no	named check or pattern: iban_check_digits, bic_length, uk_sort_code, digits(n) ...
`example`	string	no	shown on the form
`effective_from`	date	yes	
`status`	string	yes	· default **active**

- One row per rail × field, in display order. The starting templates: SWIFT — account number or IBAN, BIC, bank name and address, intermediary bank (optional); SEPA — IBAN, BIC; UK Faster Payments — sort code, account number; a local rail — bank code, account number.
- format_rule names the check applied on save: iban_check_digits, bic_length (8 or 11), uk_sort_code (6 digits), digits(n), or a pattern. A value that fails the rule cannot be saved.
- Wallets are not templated: a wallet's network and address stay on the account or endpoint row and are checksum-validated per network as before.
- Changed like any configuration: Finance proposes a row, Management approves; a retired field stops being required for new versions and stays readable on old ones.

3 · Project settings

PROJECT · Per client

One arrangement with a client: a product line with its own configuration — parties, groups, pairs, thresholds, partner configurations — and its own policies. Several projects may belong to one CLIENT (BF, BF Sub and BF Withdrawal). In it also carries the settlement policy with an exposure cap, the network-fee policy and the receiver-fee default at conversion.

Field	Type	Required	Allowed values · default · meaning
client_id	int	yes	the client this arrangement belongs to
name	string	yes	
client_label	string	no	
quote_freshness_min	int	yes	
quote_validity_min	int	yes	
grace_min	int	yes	
grace_drift_pct	decimal	yes	
late_collection_policy	string	yes	requote / honor_with_approval
bank_fee_treatment_default	string	yes	waived / carry_forward / absorbed
receiver_fee_timing	string	yes	at_conversion (default) / post_confirmation · default **at_conversion**
network_fee_policy	string	yes	absorbed / charged_back (project default; per-transaction override via FEE_DECISION). A network fee may or may not apply; when it does it is stated in USDT, when it does not it is zero and nothing posts
settlement_policy	string	yes	to_zero / hold_allowed · default **to_zero**
shortfall_tolerance	decimal	yes	· default **0**
exposure_ceiling	decimal	no	cap on balance resting at a partner
status	string	yes	
state	string	yes	draft / validated / active · default draft. Only an active project can price a deal; validated means every rule of Section 6 passes for the project and each of its partner configuration blocks; Management activates (see “Project states” in section 1)
is_test	bool	yes	default false. A dummy twin project set up from the same archetype as a real project, used only for scripted test deals on staging (Cutover Approach, Section 3); every row that belongs to it is removed by the pre-copy filter before production is loaded, and its figures are excluded from every count and balance shown to NPL

- One arrangement with a client; several projects may belong to one CLIENT. Everything the arrangement needs sits under it.
- quote_freshness_min < quote_validity_min; grace_min and grace_drift_pct govern the grace decision after validity (adds extensions before validity ends).
- network_fee_policy is the project default; a one-off override is a FEE_DECISION. When a network fee applies it is stated in USDT; when it does not, nothing posts.
- exposure_ceiling caps the balance resting at a partner (settlement_policy hold_allowed); the client-level cap is on CLIENT.

THRESHOLD · Per project

Project-level system settings: limits and escalation rules evaluated at write time. settlement_limit now also bounds balance held at a partner under settlement_policy = hold_allowed.

Field	Type	Required	Allowed values · default · meaning
project_id	int	no	reference
kind	string	yes	max_amount / settlement_limit / daily_slots / rate_variation_rel / custom; kinds added in the design review: rate_min_observations, rate_fallback_spread_abs, rate_abs_tolerance (Calculation Specification 2a), approval_time_limit (one row per approval stage, Lifecycle 8a), cutoff_warning_min, lock_idle_minutes (default 15; Lifecycle 6a) / rate_min_observations / rate_fallback_spread_abs / rate_abs_tolerance / approval_time_limit / cutoff_warning_min / lock_idle_minutes (the six added kinds are described below)
currency	string	no	
value	decimal	yes	
condition_expr	string	no	custom conditions
escalation	string	yes	finance / management

- One row per kind and currency; currency may be blank for kinds that are not amounts (daily_slots, rate_variation_rel).
- escalation names who decides when the threshold is crossed: finance or management; the escalation adds the approval, it never replaces the first.
- settlement_limit also bounds the balance held at a partner under settlement_policy hold_allowed.

Six further THRESHOLD kinds (added in the design review), used by the breach rules of the Calculation Specification (section 2a); NPL Finance sets the values, Management approves them like any threshold.

kind	currency	value	escalation	meaning
rate_min_observations	—	count (e.g. 10)	finance	fewer confirmed rate versions than this in the two-week window → the two-week average is not used; the fallback threshold applies
rate_fallback_spread_abs	—	fraction (e.g. 0.015)	finance	the absolute spread above which a rate version breaches when the two-week average is unavailable or unusable (zero or negative)
rate_abs_tolerance	—	fraction (e.g. 0.005)	finance	a spread further than this from the two-week average, in either direction, is flagged for Finance to verify before use; nothing freezes
approval_time_limit	—	minutes	finance / management	one row per stage, the stage named in condition_expr (collection_verification, rate_confirmation, conversion_approval, settlement_approval, execution_confirmation, eod_reconciliation): after this many minutes a pending item of that stage is overdue and appears on the escalation role's list (Lifecycle & Approvals, 8a)

kind	currency	value	escalation	meaning
cutoff_warning_min	—	minutes (e.g. 60; a second row 30)	finance / management	a warning is raised this many minutes before a partner's cutoff for any item still pending against it
lock_idle_minutes	—	minutes (default 15)	management	a RECORD_LOCK not touched for this long expires on its own; Management may clear one earlier (Lifecycle & Approvals, 6a)

CURRENCY_PAIR · Per project, or platform

A pair the project trades, with quote direction and rate precision. Rounding is a per-pair choice on each side: amount_rounding with rounding_unit on payouts, cash_rounding with cash_rounding_unit on the collected cash amount; what is dropped stays client money, what is added by rounding up is NPL's cost (EXP_ROUNDING). from_currency may equal to_currency: the leg still converts, at rate 1, so the fee is captured at conversion as on any other pair. Cash pairs are priced per city on the rate version and the deal, not on the pair. a pair with no project is a system reporting pair, used only to convert figures into the reporting currency.

Field	Type	Required	Allowed values · default · meaning
project_id	int	no	null when scope = platform
scope	string	yes	project / platform — a platform pair (currency → reporting currency) belongs to no project, is managed under system settings, is priced through PAIR_RATE_SOURCE like any pair and is never quoted · default **project**
from_currency	string	yes	
to_currency	string	yes	
quote_direction	string	yes	out_per_in / in_per_out · default **out_per_in**
rate_precision	int	yes	· default **5**
amount_rounding	string	yes	exact / truncate_unit / nearest_unit / round_up_unit / nearest_n / round_up_n (exact is the default for bank transfers — cents go out; the unit options apply where the pair says so, in practice to cash hand-overs) · default **exact**
cash_rounding	string	no	truncate_n / nearest_n / round_up_n — how the collected cash amount is rounded to cash_rounding_unit only)
cash_rounding_unit	int	no	100 / 500 / 1000 by currency — the unit the collected cash amount is rounded to; AED cash 100, USD cash in Dubai 10 (NPL, 9 Oct)
status	string	yes	
rounding_unit	decimal	no	required for nearest_n / round_up_n (1,000 for THB, INR, JPY payouts)

- scope project needs project_id; scope platform (currency → reporting currency) has none, is managed under system settings and is never quoted.
- amount_rounding is the payout-side menu (exact / truncate_unit / nearest_unit / round_up_unit / nearest_n / round_up_n); exact is the default for bank transfers; nearest_n and round_up_n need rounding_unit.
- cash_rounding with cash_rounding_unit applies to the collected cash amount (AED 100, USD cash in Dubai 10, 1,000 for Asian currencies); both blank for a pair that is never collected in cash.
- from_currency may equal to_currency: the leg converts at rate 1 and the fee is captured at conversion.
- A pair needs at least one PAIR_RATE_SOURCE with a unique priority before it can be used.

PAIR_RATE_SOURCE · Per pair

The prioritised source chain for a currency pair.

Field	Type	Required	Allowed values · default · meaning
currency_pair_id	int	yes	reference
rate_source_id	int	yes	reference
priority	int	yes	

- Priority 1 is tried first; the chain continues on staleness or outage.
- Platform pairs use the same chain: it is what REPORTING_VALUE converts with.

OWN_WALLET · Per project

NPL’s own wallets, by role: reroute contingency (AQN-LT-Sub at Aquanow), the own desk of the NPL-GR project, and a collection bridge — a wallet on a network a partner does not support (ERC-20 for Ali) or one that isolates risk, on which NPL collects and from which it forwards to the leg’s partner as a hop within custody_max_hours. Custody is allowed only in those three cases; the custody view is the standing control. Also receives recovered dues.: client money may rest in NPL's own wallet in four cases, not three — an approved reroute before conversion, a reroute after conversion (REROUTE.phase post_conversion), a collection bridge within custody_max_hours, and an own-desk leg; role reroute_contingency serves both reroute phases and the CUSTODY view names the reroute and its phase for every resting balance.

Field	Type	Required	Allowed values · default · meaning
project_id	int	no	reference
provider	string	yes	e.g. Aquanow
account_label	string	no	e.g. LT Sub
role	string	yes	reroute_contingency / own_desk / collection_bridge — LT Sub at Aquanow is the NPL-GR own desk; NPL’s ERC-20 wallet is a bridge for partners without that network
networks	string	no	networks the wallet receives on (ERC-20 where Ali cannot)
network	string	yes	
address	string	yes	
signer_arrangement	string	yes	hardware / multisig
custody_max_hours	int	yes	

- Optional. role says which of the three allowed custody cases the wallet serves: reroute_contingency, own_desk or collection_bridge; custody is allowed in no other case.
- networks lists the networks a bridge receives on (ERC-20 where a partner cannot).
- custody_max_hours is the limit the custody view ages against; an outbound transfer needs dual Management approval and is executed by NPL's signers outside the platform.

4 · Parties: senders, receiver groups, receivers, entities, accounts, screenings

Bank account details are rows of the database from this edition (decision of 26 October 2026): BANK_ACCOUNT_DETAILS below, one versioned row per bank account, with the fields of its rail taken from BANK_DETAILS_TEMPLATE (Section 2). The same row type serves a receiving entity's account, a partner's collection endpoint, a settlement rail, the details a partner holds under a destination approval, and a sender's source account. The restricted prefix of the file store keeps only the evidence.

SENDER · Per project

A sending party of the project (Sender A, Sender B). Which receiver groups a sender may pay is SENDER_RECEIVER_ALLOW. A sender's refunds go to a RECEIVER of kind sender_return linked to it.

Field	Type	Required	Allowed values · default · meaning
`project_id`	int	no	reference
`name`	string	yes	Sender A, Sender B
`risk_tier`	string	yes	
`status`	string	yes	

- A single paying party. Not tied to a partner.
- Must have at least one SENDER_RECEIVER_ALLOW row before it can be chosen on a deal; refunds go to its RECEIVER of kind sender_return.

SENDER_RECEIVER_ALLOW · Per sender

sendable_to: which receiver groups a sender may pay. Must pre-exist for collect-first attribution. targets the group, the unit a sender's flow is attributed to.

Field	Type	Required	Allowed values · default · meaning
`sender_id`	int	yes	reference
`receiver_group_id`	int	yes	sendable_to

- The allow-list (sendable_to). A sender may pay only the groups listed here; the group belongs to the same project.

COLLECTION_SENDING_ENTITY · Per sender

Provenance: where the sender's funds originated. Reference only; never a payout destination (a refund goes to the sender_return receiver's chosen account, not back to the source wallet). the source account, when known, is a BANK_ACCOUNT_DETAILS row.

Field	Type	Required	Allowed values · default · meaning
`sender_id`	int	yes	reference
`label`	string	yes	sender's own account or a client's account
`bank_account_details_id`	int	no	nullable; the source account when it is known

- Provenance only: where the sender's funds come from. Never a payout destination.

FEE_OVERRIDE · Per sender × partner pair

A standing sender-specific override of the fee on a partner × pair. One-off decisions on a single transaction are FEE_DECISION. may carry its own sender / receiver split instead of inheriting the structure's proportion (Evo's selected senders pay 1.0 % while Evo still pays 0.6 %).

Field	Type	Required	Allowed values · default · meaning
`sender_id`	int	yes	reference
`partner_pair_id`	int	yes	reference

Field	Type	Required	Allowed values · default · meaning
pct	decimal	yes	
effective_from	datetime	yes	
approved_by	int	yes	reference
sender_pays_pct	decimal	no	optional; with receiver_pays_pct replaces the structure's proportion
receiver_pays_pct	decimal	no	optional; both blank → structure's proportion; both set → sum = pct

- Replaces the fee structure's pct for that sender from effective_from; the structure's other terms still apply.
- sender_pays_pct and receiver_pays_pct are both blank (the structure's proportion applies) or both set and summing to pct.

RECEIVER_GROUP · Per project

The unit entitlement is tracked on (design F). Every project has one default group created with it; counterparty receivers sit in exactly one group. Payouts within a group are ordinary settlement; across groups they are offsets; with a single group no offset can arise. Groups hold counterparty receivers only: sender_return and partner_transit receivers are group-less.

Field	Type	Required	Allowed values · default · meaning
project_id	int	no	reference
name	string	yes	
is_default	bool	yes	exactly one per project
status	string	yes	

- Exactly one group per project has is_default = true; it is created with the project.
- The unit entitlement is tracked on: payouts within a group are ordinary settlement, across groups an offset needing Management approval. Refunds use group-less sender_return receivers, so no return group is needed.

RECEIVER · Per project

A party within a receiver group (Receiver X, Receiver Y) owning one or more RECEIVING_ENTITY rows — when kind = counterparty. kind sender_return stands for a SENDER so refunds use the ordinary payout path and the partner's destination approval; kind partner_transit stands for a partner's collection endpoint so the hop of a two-leg route is an ordinary payout. Both are group-less (group_id null), never appear on forward deals or allow lists, and a payout to them is attributed to the group of the leg that pays them.

Field	Type	Required	Allowed values · default · meaning
project_id	int	no	reference
group_id	int	no	receiver group; null unless kind = counterparty
kind	string	yes	counterparty / sender_return / partner_transit · default **counterparty**
sender_id	int	no	when kind = sender_return
partner_config_id	int	no	when kind = partner_transit
name	string	yes	the group: Receiver X, Receiver Y
status	string	yes	

- kind counterparty needs group_id (exactly one group); kind sender_return needs sender_id and no group; kind partner_transit needs partner_config_id and no group.
- sender_return and partner_transit receivers never appear on forward deals or allow-lists; a payout to them is attributed to the group of the leg that pays them.

RECEIVING_ENTITY · Per receiver

A member of a receiver (Entity X1, Entity X2). Bank or wallet details and currency move down to RECEIVING_ENTITY_ACCOUNT.

Field	Type	Required	Allowed values · default · meaning
receiver_id	int	yes	reference
name	string	yes	Entity X1, Entity X2
status	string	yes	

- A legal entity of the receiver. Bank and wallet details live on its accounts, not here.

RECEIVING_ENTITY_ACCOUNT · Per entity

One account of a receiving entity, with its own currency and rail (EUR via SWIFT and EUR via SEPA are two accounts). An account is a bank account or a wallet (account_kind): a bank account carries bank_account_details_id and a bank rail; a wallet carries network and address explicitly, mirroring the partner-side COLLECTION_RECEIVING_ENDPOINT, so addresses are checksum-validated, checked against the rail, and found when one address is registered with two partners. Destination approvals and settlement lines point at the account; the bank details themselves are in BANK_ACCOUNT_DETAILS, versioned.

Field	Type	Required	Allowed values · default · meaning
receiving_entity_id	int	yes	reference
currency	string	yes	
rail	string	yes	SWIFT / SEPA / TRC-20 / ERC-20 / ...
account_kind	string	yes	bank / wallet
bank_account_details_id	int	no	when account_kind = bank; the current BANK_ACCOUNT_DETAILS version
network	string	no	when account_kind = wallet
address	string	no	when account_kind = wallet; checksum-validated per network
status	string	yes	
usage	string	yes	standing / single_use — a local account the receiver gives for one payout · default **standing**
reserved_for_deal_id	int	no	nullable; the one deal a single_use account serves

- One row per currency × rail (EUR via SWIFT and EUR via SEPA are two accounts).
- account_kind bank carries bank_account_details_id — the current BANK_ACCOUNT_DETAILS version; formerly bank_details_ref, a reference into an external store — and a bank rail; the account is active only when that details row is active (rule 14); account_kind wallet carries network and address, checksum-validated per network, and rail equal to the network.
- usage single_use is a local account the receiver gives for one payout; it names reserved_for_deal_id and serves that deal only.
- An account is payable through a partner only once a DESTINATION_APPROVAL with that partner is approved.

BANK_ACCOUNT_DETAILS · Per bank account, versioned

The bank details of one account, in the database: a receiving-entity account, a partner's collection endpoint, a settlement rail, the details a partner holds under a destination approval, or a sender's source account. Versioned: a change is a new row that names the row it supersedes (supersedes_id); the old row is kept and is what any settlement line released before the change points at through its account. Common fields here; the rail-specific values (IBAN, BIC, sort code, bank code ...) are stored under rail_fields, keyed by the field_key of the rail's BANK_DETAILS_TEMPLATE rows and validated against them on save. Entered by Operations, approved by Finance or Management (controlled action bank_details_approve); read in full by Operations, Finance and Management, with every full read written to AUDIT_LOG. The evidence (bank letter, KYC, legal document) stays a file under the restricted prefix (evidence_ref).

Field	Type	Required	Allowed values · default · meaning
supersedes_id	int	no	nullable; the BANK_ACCOUNT_DETAILS row this version replaces
holder_name	string	yes	the account holder as the bank knows it
bank_name	string	yes	
country	string	yes	ISO 3166 alpha-2
currency	string	yes	
rail	string	yes	names the BANK_DETAILS_TEMPLATE rows that apply
rail_fields	json	yes	the rail-specific values, keyed by the template's field_key; validated by format_rule on save
evidence_ref	string	yes	bank letter or confirmation, restricted prefix of the file store
entered_by	int	yes	Operations user
entered_at	datetime	yes	
status	string	yes	· default **draft**

- Entered by Operations from the rail's template: choose the rail, fill the template's fields, attach the bank letter or confirmation as evidence (a file under the restricted prefix). Saved only when every required field is present and every format rule passes.
- Approved by Finance or Management (controlled action bank_details_approve, approver never the person who entered it); the row becomes active and the account, endpoint, rail or destination approval that points at it may become active.
- Never edited: a change is a new row with supersedes_id naming the old one; the old row becomes superseded and stays. The account keeps pointing at the current version; a settlement line released earlier still resolves to the version current at its release.
- Read in full by Operations, Finance and Management — no masking anywhere, one card component shows the common fields then the rail's fields in template order. Every full read is written to AUDIT_LOG (Roles & Visibility rule 10).

WALLET_SCREENING · Per project, per address and day

A screening of one wallet address on one day — a sender's declared sending wallet before a collection, or a receiver's wallet before a settlement — with the result, the decision and who made it. Operations records it as part of transaction coordination; a reject may be overridden only by Finance. The decision picks the endpoint: accept_alternate lands only on an elevated-band endpoint.

Field	Type	Required	Allowed values · default · meaning
project_id	int	yes	reference
subject_kind	string	yes	sender_wallet / receiver_wallet
sender_id	int	no	when subject_kind = sender_wallet
receiving_entity_account_id	int	no	when subject_kind = receiver_wallet
network	string	yes	
address	string	yes	
provider_ref	string	no	screening tool reference
result	string	yes	standard / elevated / unacceptable
decision	string	yes	accept / accept_alternate / reject
screened_by	int	yes	Operations
screened_at	datetime	yes	
evidence_ref	string	no	

- subject_kind sender_wallet names sender_id; receiver_wallet names receiving_entity_account_id.
- decision accept lands on a standard endpoint; accept_alternate only on an endpoint with risk_band elevated; reject stops the collection unless Finance overrides it (ROLE_RIGHT override_screening_reject).
- Operations records it; it is configuration-domain data because the decision binds later deals on the same address.

5 · The partner configuration block

PARTNER_CONFIG · Per project × partner

How a partner is used in this project. Fees, vehicles, endpoints, destination approvals and rebate rules hang off it. applicable_for_reroute defines the alternate-partner set. an own-desk configuration is NPL itself as the converting party; its PARTNER_PAIR rows are priced fee_on_market with NPL's markup and its endpoints are NPL's own wallets. in NPL-GR the own desk holds NPL-GR's remittance balances as ordinary partner-coded client accounts (stands); the reseller books are separate EUR postings.

Field	Type	Required	Allowed values · default · meaning
`project_id`	int	yes	reference
`partner_id`	int	yes	reference
`applicable_for_reroute`	bool	yes	· default **false**
`status`	string	yes	
`is_own_desk`	bool	no	default false; NPL converts in its own wallet for this project (NPL-GR); enabling needs Management approval · default **false**

- How this partner is used in this project. Pairs, fee structures, vehicles, endpoints, destination approvals, markup-share and rebate rules hang off it.
- applicable_for_reroute puts the partner in the project's alternate set (Management-approved).
- is_own_desk true means NPL itself converts in its own wallet for this project: its pairs are priced fee_on_market with NPL's markup and its endpoints point at an OWN_WALLET of role own_desk; enabling needs Management approval (enable_own_desk).

PARTNER_PAIR · Per partner config × pair

A pair handled by a partner in this project. Fees are per partner × pair, so choosing the partner is part of pricing.

Field	Type	Required	Allowed values · default · meaning
`partner_config_id`	int	yes	reference
`currency_pair_id`	int	yes	reference
`partner_pricing`	string	yes	fee_on_market / fee_in_rate
`partner_fee_pct`	decimal	no	known only for fee_on_market
`status`	string	yes	
`needs_bank_lock`	bool	no	the pair relies on the banking channel, so the partner's cutoff governs its rate lock and its settlement (Evo pairs into EUR; USDT → USD / SGD at Ali); crypto-only pairs do not · default **false**

- currency_pair_id is one of the project's pairs (scope project).
- partner_pricing agrees with the partner's disclosure_style; partner_fee_pct is set exactly when partner_pricing is fee_on_market.
- needs_bank_lock true means the partner's cutoff governs the rate lock and the settlement on this pair; it needs PARTNER.bank_cutoff_time.

FEE_STRUCTURE · Per partner pair, versioned

The versioned NPL client-side fee for a partner × pair. Stamped on the deal group at pricing. the sender and receiver parts are each a percentage of the principal at the basis rate, never compounded — fee_sender_part = amount_in × rate × sender_pays_pct, fee_receiver_part = amount_in × rate × receiver_pays_pct, client_net = amount_in × rate × (1 – sender_pays_pct – receiver_pays_pct); the partner's cost then comes out of NPL's parts (fee_in_rate) or is booked separately (fee_on_market). Evo USDT → EUR: sender pays 1 % or 1.4 % by sender type, receiver always 0.6 %; EUR → EUR 2.8 % split 1.4 / 1.4. Target-out quotes apply the fee as a markup on the target: amount_in = target × (1 + sender_pays_pct) ÷ rate.

Field	Type	Required	Allowed values · default · meaning
partner_pair_id	int	yes	reference
rate_basis	string	yes	market / agent_rate / partner)
mode	string	yes	fixed / variable · default **fixed**
pct	decimal	yes	
sender_pays_pct	decimal	yes	
receiver_pays_pct	decimal	yes	
min_markup_pct	decimal	no	
max_markup_pct	decimal	no	
min_margin_pct	decimal	yes	
variance_threshold_rel	decimal	yes	
quoting_rate_policy	string	yes	worst_applicable / designated / best_applicable · default **worst_applicable**
effective_from	datetime	yes	
customer_facing_fee_basis	string	yes	actual / sender_pays_on_agent_rate / fixed · default **actual**
customer_facing_fee_pct	decimal	no	when customer_facing_fee_basis = fixed (BF via Ali shows 1 %)

- sender_pays_pct + receiver_pays_pct = pct. Each part is a percentage of the principal at the basis rate, never compounded.
- rate_basis market prices on the reference feed; agent_rate prices on the partner's rate version.
- mode variable needs min_markup_pct ≤ max_markup_pct; a markup above max_markup_pct is a Management decision (raise_fee_cap).
- customer_facing_fee_basis fixed needs customer_facing_fee_pct (what the quote prints); the figures are derived, never typed.
- A new version is a new row with a later effective_from; a deal keeps the version it was priced on.

FEE_TIER · Per fee structure

A volume tier of a fee structure (GDC: 2.5 % from USD 0, 2.25 % from USD 500,000 in the calendar month). A structure with tiers has one tier at volume 0. The tier applied is chosen at quote from the project's month-to-date Σ amount_in; a deal that crosses a threshold is split across two tiers (DEAL_FEE_TIER).

Field	Type	Required	Allowed values · default · meaning
fee_structure_id	int	yes	reference
min_monthly_volume	decimal	yes	threshold in volume_currency, per calendar month
volume_currency	string	yes	
pct	decimal	yes	
sender_pays_pct	decimal	no	optional
receiver_pays_pct	decimal	no	optional
effective_from	datetime	yes	

- A structure with tiers has exactly one tier at min_monthly_volume 0; thresholds are unique within a structure.
- The tier is chosen at quote from the project's month-to-date Σ amount_in (PROJECT_MONTH_VOLUME); a deal that crosses a threshold is split across two tiers (DEAL_FEE_TIER).
- sender_pays_pct / receiver_pays_pct are optional; both blank means the structure's proportion.

PARTNER_ENTITY · Per partner config

The partner’s legal vehicle (PT-tour, PT Global Inc): the name that appears on paperwork, the partner-side mirror of RECEIVING_ENTITY. Fixed on a deal leg at quote; only a REROUTE changes it. Owns the partner’s collection endpoints.

Field	Type	Required	Allowed values · default · meaning
partner_config_id	int	yes	reference
name	string	yes	the partner’s legal vehicle: PT-tour, PT Global Inc
status	string	yes	

- The partner's legal vehicle (PT-tour, PT Global Inc), the name that appears on paperwork. Fixed on a deal leg at quote; only a REROUTE changes it.
- Owns the partner's collection endpoints.

COLLECTION_RECEIVING_ENDPOINT · Per partner entity

One collection endpoint at the partner, owned by a partner entity: bank-in or crypto-in by kind, with the same field shape as RECEIVING_ENTITY_ACCOUNT (rail required only on active bank endpoints). rail is nullable — null means not yet known, never doesn’t matter. A bank endpoint cannot become active, and no COLLECTION may reference it, until rail is set; a wallet endpoint may leave rail null, and when set it must match network. the endpoint’s kind must match the leg’s collection_method — wallet for crypto-in, bank for bank-in, none for cash.

Field	Type	Required	Allowed values · default · meaning
partner_entity_id	int	yes	the vehicle whose name is on the account or wallet
kind	string	yes	bank / wallet
currency	string	yes	
rail	string	no	nullable: a bank endpoint needs it to go active; optional for wallets, must match network when set
bank_account_details_id	int	no	when kind = bank; the current BANK_ACCOUNT_DETAILS version
network	string	no	TRC-20 / ERC-20 / …; when kind = wallet
address	string	no	when kind = wallet; checksum-validated per network
status	string	yes	
own_wallet_id	int	no	nullable; set when the endpoint is NPL’s own wallet — under an own-desk configuration or as a collection bridge for a partner that lacks the network or for risk isolation; the endpoint still belongs to the leg’s partner configuration
risk_band	string	yes	standard / elevated — which screening decisions may land here · default **standard**
usage	string	yes	standing / single_use (ad hoc local deposit accounts) · default **standing**
reserved_for_deal_id	int	no	nullable; the one deal a single_use endpoint serves
expected_amount	decimal	no	nullable; the amount a single_use endpoint for

- kind bank carries bank_account_details_id (the current BANK_ACCOUNT_DETAILS version; formerly bank_details_ref) and needs rail before it can become active (null means not yet known), and the details row must be active (rule 14); kind wallet carries network and address, and rail, when set, equals network.
- A wallet endpoint's network must be one the partner offers; cash and local_deposit collections have no standing endpoint.
- own_wallet_id names an OWN_WALLET when the endpoint is NPL's own wallet: under an own-desk configuration or as a collection bridge; the endpoint still belongs to the leg's partner configuration.
- usage single_use names reserved_for_deal_id and expected_amount (ad hoc local deposit accounts).

- The endpoint's kind must match the leg's collection method; the system refuses a collection on an endpoint of another partner entity.

SETTLEMENT_SENDING_ENTITY · Per partner config

The narrative on an outgoing transfer: what the receiver sees. Holds no money.

Field	Type	Required	Allowed values · default · meaning
partner_config_id	int	yes	reference
narrative_name	string	yes	what the receiver sees
status	string	yes	

- The name the receiver sees on an outgoing transfer. Holds no money.

SETTLEMENT_RAIL · Per partner config

The bank or EMI that executes an outgoing transfer. Holds no money. Carries the expected bank fee. its account's details are a BANK_ACCOUNT_DETAILS row.

Field	Type	Required	Allowed values · default · meaning
partner_config_id	int	yes	reference
name	string	yes	bank or EMI executing the transfer
bank_account_details_id	int	yes	the rail's own account
expected_bank_fee	decimal	no	
currency	string	yes	

- The bank or EMI that executes the transfer. expected_bank_fee is the figure the bank-fee true-up compares against (BANK_FEE_EVENT).
- bank_account_details_id names the rail's own account as a BANK_ACCOUNT_DETAILS row (formerly bank_details_ref); the rail is usable only while that row is active (rule 14).

DESTINATION_APPROVAL · Per partner config × account

Destination approval per partner: a receiving-entity account as registered with this partner, with the partner's approval status and the payout details the partner holds. The details follow the account's kind: bank_account_details_id when the account is a bank account; network and address as registered with the partner when it is a wallet.

Field	Type	Required	Allowed values · default · meaning
partner_config_id	int	yes	reference
receiving_entity_account_id	int	yes	
approval_status	string	yes	pending_partner / approved / suspended · default **pending_partner**
bank_account_details_id	int	no	the bank details as registered with this partner, when the account is a bank account
network	string	no	as registered with this partner; only when the account's kind is wallet
address	string	no	as registered with this partner; only when the account's kind is wallet
approved_at	datetime	no	
wallet_screening_id	int	no	nullable; the receiver-side screening of a wallet account

- Destination approval per partner: an account as registered with this partner.
- The payout details follow the account's kind: bank_account_details_id — the details as registered with this partner, a BANK_ACCOUNT_DETAILS row (formerly bank_details_ref) — for a bank account; network and address as

registered with the partner for a wallet; a wallet account carries its WALLET_SCREENING.

- A quote-first deal leaves Inquiry only if at least one account in the entitled group is approved with the chosen partner.

MARKUP_SHARE_RULE · Per party × project (optionally one partner pair or fee structure)

How an markup-share party is paid on a project (and optionally only on one partner × pair): a share of NPL's earnings in a ratio, or a fixed % applied inside the sender-facing markup (§4.2a). also the customer who provides a sub-account and shares NPL's markup (Raeen): basis share_of_net_markup, several rules with different shares on one project, and settlement by the party itself when it holds the converted money.

Field	Type	Required	Allowed values · default · meaning
markup_share_party_id	int	yes	reference
project_id	int	yes	reference
partner_pair_id	int	no	optional
basis	string	yes	share_of_earnings / fixed_pct / share_of_net_markup (base = fee parts − partner cost on the leg, on amount × market; rebates never in the base)
pct	decimal	yes	
effective_from	datetime	yes	
fee_structure_id	int	no	optional scope: only deals priced on this structure (Raeen sub-account 50 %, Raeen direct downlines 75 %)
party_kind	string	yes	markup-share party / client_party
settlement_mode	string	yes	npl_pays / party_retains — the party holds the converted funds and pays NPL the remainder monthly · default **npl_pays**

- basis share_of_earnings or share_of_net_markup is a ratio of NPL's earnings on the leg (base = fee parts − partner cost; rebates never in the base); fixed_pct is applied inside the sender-facing markup.
- Several rules on one project are allowed; Σ shares of the same base ≤ 100 %.
- settlement_mode party_retains: the party holds the converted funds and pays NPL the remainder monthly.

PARTNER_REBATE_RULE · Per partner config

A partner rebating part of its fee to NPL (§4.2b: Jeton takes 1 % or 1.25 % and pays 0.4 % of the amount back; 0.3 % on Novi's 0.9 %; Aquanow 0.3 % on 0.8 %). The rule is a percentage of the amount (basis pct_of_amount; Reconciled monthly through PARTNER_REBATE_ACCRUAL against the partner statement, which covers NPLify deals only. The rebate is NPL's alone — never shared with an markup-share party or client party, never inside the share base — and is reported as its own stream (EARN_REBATE).

Field	Type	Required	Allowed values · default · meaning
partner_config_id	int	yes	reference
effective_from	datetime	yes	
basis	string	yes	pct_of_amount — the rebate is a percentage of the amount converted (Jeton 0.4 % of amount on a 1 % or 1.25 % fee, Novi 0.3 %, Aquanow 0.3 %) · default **pct_of_amount**
pct	decimal	yes	the rebate in the chosen basis
fee_structure_id	int	no	optional scope
sender_id	int	no	optional scope (Novi)
payout_currency	string	yes	the currency the partner pays the rebate in and the accrual is expressed in — the amount currency

- Only for partners that return part of their fee. basis is pct_of_amount: the rebate is a percentage of the amount converted.

- payout_currency is the currency the partner pays the rebate in and the accrual is expressed in — the amount currency.
- Optional scope by fee_structure_id or sender_id. The rebate is NPL's alone (EARN_REBATE): never shared, never in a markup-share base; reconciled monthly through PARTNER_REBATE_ACCRUAL.

Rebates and the percentages in this document. The Jeton rebate of 0.4 % quoted in the worked example and in the Scenario Catalogue is not a fixed value: it is a PARTNER_REBATE_RULE row of the project's partner configuration, changed by a new version with a later effective_from like any fee, and NPL confirmed on 20 October that it is set per project, not per deal. Every percentage, rate and amount in Sections 2 to 7 is illustrative, taken from NPL's fee table of 8 October 2026, and is replaced by the project's configured values; none is written into code.

6 · Cross-entity rules the system enforces on save

1. Every project belongs to one client; a project's exposure_ceiling never exceeds its client's; both are compared through REPORTING_VALUE, so a platform pair must exist for every currency that rests at a partner.
2. Every project has exactly one default receiver group, created by the system. A counterparty receiver is in exactly one group; sender_return and partner_transit receivers are group-less and never on a forward deal or an allow-list.
3. Every sender has an allow-list naming groups of the same project and a sender_return receiver for refunds; a sender with no allow-list cannot be chosen on a deal.
4. Every receiving entity has at least one account; accounts are unique per entity, currency and rail; a bank account names its BANK_ACCOUNT_DETAILS row, a wallet account a network and a checksum-valid address with rail = network. A single-use account or endpoint names the one deal it serves.
5. A partner pair's currency pair is a project pair; partner_pricing agrees with the partner's disclosure_style; partner_fee_pct is present exactly for fee_on_market; needs_bank_lock requires the partner's bank_cutoff_time and cutoff_timezone.
6. A fee structure's sender and receiver parts add up to its pct; min_markup ≤ max_markup; a fixed customer-facing fee names its pct; tiers have exactly one row at volume 0; a new version is a new row with a later effective_from.
7. A partner configuration that will pay out needs at least one partner entity, one settlement sending entity (narrative) and one settlement rail. A bank endpoint cannot go active without a rail; a wallet endpoint's network is one the partner offers; an own-desk configuration needs an own wallet of role own_desk.
8. A currency pair needs at least one rate source with a unique priority before it is usable; nearest_n and round_up_n need a rounding unit; cash rounding needs its unit. A platform pair has no project and ends in the reporting currency.
9. A destination approval points at an account of a receiving entity in the same project; a quote-first deal for a group needs at least one approved destination with the chosen partner. A wallet account's approval carries its screening.
10. A quote template prints sender-facing fields only; a screening decision of accept_alternate lands only on an elevated-band endpoint; a cutoff override is a one-day or a weekday rule, never both; an extension names the partner's agreement.
11. quote_freshness_min is shorter than quote_validity_min; all percentages are stored as decimals; Σ markup shares of one base ≤ 100 %; a rebate rule is pct_of_amount with a payout currency.
12. applicable_for_reroute, is_own_desk, exposure ceilings, thresholds, fee structures and tiers, templates, system settings and platform pairs are Management-approved changes; users and roles

need a second Management user.

13. A project is validated only when rules 1 to 12 pass for it and for each of its partner configuration blocks; only a validated project can be activated, and only Management activates it. A partner configuration block added to an active project cannot be used on a deal until the rules pass for it.
14. A bank account, a bank endpoint and a settlement rail name a BANK_ACCOUNT_DETAILS row of the same currency and rail and can be active only while that row is active; a row is saved only when the rail's template is satisfied (required fields present, format rules passed: IBAN check digits, BIC length, sort-code pattern) and its evidence is attached; a change to any detail is a new row that supersedes the old one, never an edit; a details row is approved by Finance or Management, never by the Operations user who entered it.

7 · Worked example: the FRS project, end to end

The FRS describes one client flow: Sender A and Sender B pay USDT, partners Ali (through its PT vehicles) or Jeton convert to EUR, and Receiver X (Entity X1, Entity X2) and Receiver Y are paid in euros. Below is that project expressed in the schema. Names, policies and the fee split come from NPL's documents; rates of fee, thresholds, holidays, addresses and reference numbers are **illustrative placeholders** to be replaced with NPL's values.

How to read it: one block per entity type, in the order of Sections 2 to 5. A code such as `DEFAULT` or `X1-EUR-SEPA` is how one row refers to another; the system assigns the numeric ids. Jeton's pending destination for Receiver Y and Sender B's fee override are there on purpose, to show a rule at work.

```
system_settings:
 - key: reporting_currency
 value: USD
 effective_from: "2026-10-01"
 - key: reporting_rate_max_age_min
 value: "60"
 effective_from: "2026-10-01"
 - key: operating_timezone
 value: Asia/Hong_Kong
 effective_from: "2026-10-01"
platform_pairs:
 - code: USDT-USD
 from_currency: USDT
 to_currency: USD
 quote_direction: out_per_in
 rate_precision: 5
 amount_rounding: exact
 status: active
 sources:
 - rate_source: KRAKEN
 priority: 1
 - rate_source: CMC
 priority: 2
 - code: EUR-USD
 from_currency: EUR
 to_currency: USD
 quote_direction: out_per_in
 rate_precision: 5
 amount_rounding: exact
 status: active
 sources:
 - rate_source: OANDA
 priority: 1
 - rate_source: XE
 priority: 2
rate_sources:
 - code: KRAKEN
 staleness_threshold_s: 120
 - code: CMC
 staleness_threshold_s: 300
 - code: OANDA
 staleness_threshold_s: 300
 - code: XE
 staleness_threshold_s: 300
partners:
 - code: ALI
 name: Ali
 disclosure_style: fee_in_rate
 rate_expiry_time: "17:00 local"
 holiday_calendar: ID banking holidays
```

```
  supported_methods: [crypto, bank, cash, local_deposit]
  bank_cutoff_time: "14:30"
  cutoff_timezone: Asia/Hong_Kong
  status: active
  holidays:
  - date: "2026-12-25"
  name: Christmas Day
  scope: bank_channel
  source: public calendar
  - date: "2027-03-20"
  name: Idul Fitri (illustrative)
  scope: bank_channel
  source: partner notice
  cutoff_overrides:
  - weekday: Friday
  cutoff_time: "13:30"
  kind: early_close
  reason: pre-weekend
  recorded_by: finance
  - code: JETON
  name: Jeton
  disclosure_style: fee_on_market
  rate_expiry_time: "16:00 local"
  holiday_calendar: UK banking holidays
  supported_methods: [crypto, bank]
  bank_cutoff_time: null
  cutoff_timezone: Europe/London
  status: active
  holidays: []
  cutoff_overrides: []
markup_share_parties: []
templates:
  - code: quote
  version: 1
  channel: pdf
  language: en
  body_ref: tpl-quote-en-v1
  printed_fields: [sender_rate, customer_facing_fee_pct, customer_facing_rate,
printed_amount_out, effective_valid_until]
  effective_from: "2026-10-01"
  status: active
  - code: payment_instructions
  version: 1
  channel: email
  language: en
  body_ref: tpl-payin-en-v1
  printed_fields: [amount_in, network, address, effective_valid_until]
  effective_from: "2026-10-01"
  status: active
client:
  code: CLIENT1
  name: Client 1
  category: FRS
  legal_entity_ref: le-client1
  kyc_ref: kyc-client1
  exposure_ceiling: 400000
  status: active
project:
  code: CLIENT1-MAIN
  client: CLIENT1
  name: Client 1 (FRS project)
  client_label: Client 1
  quote_freshness_min: 5
  quote_validity_min: 60
  grace_min: 15
  grace_drift_pct: 0.001
```

```
  late_collection_policy: requote
  bank_fee_treatment_default: carry_forward
  receiver_fee_timing: at_conversion
  network_fee_policy: absorbed
  settlement_policy: to_zero
  shortfall_tolerance: 0
  exposure_ceiling: 250000
  status: active
 thresholds:
  - kind: max_amount
  currency: USDT
  value: 100000
  escalation: finance
  - kind: settlement_limit
  currency: EUR
  value: 50000
  escalation: management
  - kind: daily_slots
  value: 2
  escalation: management
  - kind: rate_variation_rel
  value: 0.002
  escalation: finance
 currency_pairs:
  - code: USDT-EUR
  from_currency: USDT
  to_currency: EUR
  quote_direction: out_per_in
  rate_precision: 5
  amount_rounding: exact
  status: active
  sources:
  - rate_source: KRAKEN
  priority: 1
  - rate_source: CMC
  priority: 2
 own_wallets:
  - code: AQN-LT-SUB
  provider: Aquanow
  account_label: LT Sub
  role: reroute_contingency
  networks: [TRC-20]
  network: TRC-20
  address: T…(wallet address)
  signer_arrangement: multisig
  custody_max_hours: 48
 bank_account_details:
  - code: BAD-X1-EUR
  holder_name: Entity X1 Ltd
  bank_name: Bank X
  country: DE
  currency: EUR
  rail: SEPA
  rail_fields:
  iban: DE89 3704 0044 0532 0130 00
  bic: COBADEFFXXX
  evidence_ref: restricted/bank-letters/x1-eur.pdf
  entered_by: ops.user
  entered_at: "2026-09-18"
  status: active
  - code: BAD-X2-EUR
  holder_name: Entity X2 GmbH
  bank_name: Bank X
  country: DE
  currency: EUR
  rail: SEPA
```

```
  rail_fields:
  iban: DE02 1203 0000 0000 2020 51
  bic: BYLADEM1001
  evidence_ref: restricted/bank-letters/x2-eur.pdf
  entered_by: ops.user
  entered_at: "2026-09-18"
  status: active
  - code: BAD-Y-EUR
  holder_name: Entity Y SA
  bank_name: Bank Y
  country: FR
  currency: EUR
  rail: SWIFT
  rail_fields:
  iban: FR14 2004 1010 0505 0001 3M02 606
  bic: BNPAFRPPXXX
  bank_address: "16 bd des Italiens, Paris"
  intermediary_bic: ""
  evidence_ref: restricted/bank-letters/y-eur.pdf
  entered_by: ops.user
  entered_at: "2026-09-18"
  status: active
  - code: BAD-PTG-IDR
  holder_name: PT Global Inc
  bank_name: Bank Mandiri
  country: ID
  currency: IDR
  rail: local
  rail_fields:
  bank_code: "008"
  account_number: "1234567890"
  evidence_ref: restricted/bank-letters/ptg-idr.pdf
  entered_by: ops.user
  entered_at: "2026-09-18"
  status: active
  - code: BAD-ALI-EUR-RAIL
  holder_name: Ali EU EMI
  bank_name: EMI Bank
  country: LT
  currency: EUR
  rail: SEPA
  rail_fields:
  iban: LT12 1000 0111 0100 1000
  bic: EMIBLT22
  evidence_ref: restricted/bank-letters/ali-eur.pdf
  entered_by: ops.user
  entered_at: "2026-09-18"
  status: active
  - code: BAD-JETON-EUR-RAIL
  holder_name: Jeton EMI
  bank_name: Jeton Bank
  country: GB
  currency: EUR
  rail: SEPA
  rail_fields:
  iban: GB29 NWBK 6016 1331 9268 19
  bic: NWBKGB2L
  evidence_ref: restricted/bank-letters/jeton-eur.pdf
  entered_by: ops.user
  entered_at: "2026-09-18"
  status: active
receiver_groups:
  - code: DEFAULT
  name: Default group
  is_default: true
  status: active
```

```
receivers:
 - code: RCV_X
 kind: counterparty
 group: DEFAULT
 name: Receiver X
 status: active
 entities:
 - code: ENTITY_X1
 name: Entity X1
 status: active
 accounts:
 - code: X1-EUR-SEPA
 currency: EUR
 rail: SEPA
 account_kind: bank
 bank_account_details: BAD-X1-EUR
 usage: standing
 status: active
 - code: ENTITY_X2
 name: Entity X2
 status: active
 accounts:
 - code: X2-EUR-SEPA
 currency: EUR
 rail: SEPA
 account_kind: bank
 bank_account_details: BAD-X2-EUR
 usage: standing
 status: active
 - code: RCV_Y
 kind: counterparty
 group: DEFAULT
 name: Receiver Y
 status: active
 entities:
 - code: ENTITY_Y
 name: Entity Y
 status: active
 accounts:
 - code: Y-EUR-SWIFT
 currency: EUR
 rail: SWIFT
 account_kind: bank
 bank_account_details: BAD-Y-EUR
 usage: standing
 status: active
 - code: RET_SND_A
 kind: sender_return
 sender: SND_A
 name: Sender A (returns)
 status: active
 entities:
 - code: RET_A
 name: Sender A
 status: active
 accounts:
 - code: RET-A-USDT-TRC20
 currency: USDT
 rail: TRC-20
 account_kind: wallet
 network: TRC-20
 address: T…(Sender A return wallet)
 usage: standing
 status: active
 - code: RET_SND_B
 kind: sender_return
```

```
  sender: SND_B
  name: Sender B (returns)
  status: active
  entities:
  - code: RET_B
  name: Sender B
  status: active
  accounts:
  - code: RET-B-USDT-TRC20
  currency: USDT
  rail: TRC-20
  account_kind: wallet
  network: TRC-20
  address: T…(Sender B return wallet)
  usage: standing
  status: active
 senders:
  - code: SND_A
  name: Sender A
  risk_tier: standard
  status: active
  sendable_to: [DEFAULT]
  provenance:
  - label: own wallet (TRC-20)
  bank_account_details: null
  fee_overrides: []
  - code: SND_B
  name: Sender B
  risk_tier: standard
  status: active
  sendable_to: [DEFAULT]
  provenance:
  - label: own wallet (TRC-20)
  bank_account_details: null
  fee_overrides:
  - partner_pair: CLIENT1-ALI/USDT-EUR
  pct: 0.012
  sender_pays_pct: 0.006
  receiver_pays_pct: 0.006
  effective_from: "2026-10-01"
  approved_by: management
 partner_configs:
  - code: CLIENT1-ALI
  partner: ALI
  applicable_for_reroute: false
  is_own_desk: false
  status: active
  pairs:
  - currency_pair: USDT-EUR
  partner_pricing: fee_in_rate
  needs_bank_lock: true
  status: active
  fee_structures:
  - rate_basis: market
  mode: fixed
  pct: 0.01
  sender_pays_pct: 0.004
  receiver_pays_pct: 0.006
  min_markup_pct: 0.008
  max_markup_pct: 0.015
  min_margin_pct: 0.003
  variance_threshold_rel: 0.2
  quoting_rate_policy: worst_applicable
  customer_facing_fee_basis: actual
  effective_from: "2026-10-01"
  tiers: []
```

```
partner_entities:
- code: PT-TOUR
name: PT-tour
status: active
endpoints:
- code: PT-TOUR-USDT-TRC20
kind: wallet
currency: USDT
rail: TRC-20
network: TRC-20
address: T…(PT-tour wallet)
risk_band: standard
usage: standing
status: active
- code: PT-GLOBAL
name: PT Global Inc
status: active
endpoints:
- code: PT-GLOBAL-USDT-TRC20
kind: wallet
currency: USDT
rail: TRC-20
network: TRC-20
address: T…(PT Global wallet)
risk_band: elevated
usage: standing
status: active
- code: PT-GLOBAL-IDR-BANK
kind: bank
currency: IDR
rail: local
bank_account_details: BAD-PTG-IDR
risk_band: standard
usage: standing
status: active
settlement_sending_entities:
- narrative_name: PT Global Inc
status: active
settlement_rails:
- name: Ali's EU bank / EMI
bank_account_details: BAD-ALI-EUR-RAIL
expected_bank_fee: 25
currency: EUR
destination_approvals:
- account: X1-EUR-SEPA
approval_status: approved
approved_at: "2026-09-20"
- account: X2-EUR-SEPA
approval_status: approved
approved_at: "2026-09-20"
- account: Y-EUR-SWIFT
approval_status: approved
approved_at: "2026-09-20"
- account: RET-A-USDT-TRC20
approval_status: approved
approved_at: "2026-09-20"
- account: RET-B-USDT-TRC20
approval_status: approved
approved_at: "2026-09-20"
markup_share_rules: []
rebate_rule: null
- code: CLIENT1-JETON
partner: JETON
applicable_for_reroute: true
is_own_desk: false
status: active
```

```
pairs:
- currency_pair: USDT-EUR
partner_pricing: fee_on_market
partner_fee_pct: 0.005
needs_bank_lock: false
status: active
fee_structures:
- rate_basis: market
mode: fixed
pct: 0.01
sender_pays_pct: 0.004
receiver_pays_pct: 0.006
min_markup_pct: 0.008
max_markup_pct: 0.015
min_margin_pct: 0.003
variance_threshold_rel: 0.2
quoting_rate_policy: worst_applicable
customer_facing_fee_basis: actual
effective_from: "2026-10-01"
tiers: []
partner_entities:
- code: JETON-EMI
name: Jeton EMI
status: active
endpoints:
- code: JETON-USDT-TRC20
kind: wallet
currency: USDT
rail: TRC-20
network: TRC-20
address: T…(Jeton wallet)
risk_band: standard
usage: standing
status: active
settlement_sending_entities:
- narrative_name: Jeton
status: active
settlement_rails:
- name: Jeton EMI (SEPA)
bank_account_details: BAD-JETON-EUR-RAIL
expected_bank_fee: 0
currency: EUR
destination_approvals:
- account: X1-EUR-SEPA
approval_status: approved
approved_at: "2026-09-22"
- account: X2-EUR-SEPA
approval_status: approved
approved_at: "2026-09-22"
- account: Y-EUR-SWIFT
approval_status: pending_partner
- account: RET-A-USDT-TRC20
approval_status: approved
approved_at: "2026-09-22"
- account: RET-B-USDT-TRC20
approval_status: approved
approved_at: "2026-09-22"
markup_share_rules: []
rebate_rule:
basis: pct_of_amount
pct: 0.004
payout_currency: USDT
effective_from: "2026-10-01"
```

What each part does in one deal

Step of the deal	Configuration it reads
Sender A asks to send USDT for Receiver Y	SENDER Sender A; the allow-list names the default group, so the deal is attributed to `DEFAULT`; Receiver Y is a counterparty receiver in it.
Operations picks Ali for USDT → EUR	PARTNER_CONFIG CLIENT1-ALI, PARTNER_PAIR USDT-EUR (fee_in_rate: Ali's margin is inside its rate; needs_bank_lock, so Ali's 14:30 Hong Kong (GMT+8) cutoff governs the rate lock), its current FEE_STRUCTURE: basis market, fixed 1.00 % (0.40 % sender, 0.60 % receiver), customer-facing figures as actual. Jeton, the alternative, is fee_on_market at 0.50 % and binds no cutoff.
Sender B instead of Sender A	FEE_OVERRIDE: 1.20 % split 0.60 / 0.60 on CLIENT1-ALI/USDT-EUR from 1 October, Management-approved; the structure's other terms still apply.
Market rate	CURRENCY_PAIR USDT-EUR, source chain Kraken then CMC, 5 decimals; a snapshot older than 120 s is not used; both feeds down → manual rate with Finance approval.
Quote clocks and the package	PROJECT: freshness 5 min, validity 60 min, grace 15 min if drift ≤ 0.10 %; beyond grace the policy is re-quote; an extension before validity ends is a QUOTE_VALIDITY_EXTENSION. The package is rendered from MESSAGE_TEMPLATE quote v1, which prints the five sender-facing fields only.
Can the quote leave Inquiry?	DESTINATION_APPROVAL: Receiver Y's account Y-EUR-SWIFT is approved with Ali, so yes. Through Jeton it is still pending_partner, so a quote-first deal for Receiver Y cannot be routed through Jeton until the partner approves — while Receiver X could.
Where does Sender A pay?	PARTNER_ENTITY PT-tour, its TRC-20 USDT COLLECTION_RECEIVING_ENDPOINT (standard band). Sender A's wallet is screened first (WALLET_SCREENING); an accept_alternate decision would land on PT Global's elevated-band wallet instead. Provenance is recorded against his COLLECTION_SENDING_ENTITY.
Conversion	Receiver part taken at conversion (receiver_fee_timing); Converting may not start before conversion_due_date — the next Hong Kong business day after a post-cutoff lock, skipping Ali's HOLIDAY rows and the Friday early close; variance tolerance 0.20 % (THRESHOLD rate_variation_rel).
Payout	SETTLEMENT from Ali through SETTLEMENT_RAIL "Ali's EU bank / EMI" under the narrative "PT Global Inc" to Y-EUR-SWIFT; amount_rounding exact (cents go out); settlement_policy to_zero; expected bank fee 25 EUR, treatment carry_forward; Management approval above 50,000 EUR or for a third same-day slot.
If Ali is unavailable	REROUTE to Jeton (applicable_for_reroute = true) via OWN_WALLET AQN-LT-SUB (role reroute_contingency), custody limit 48 hours; Jeton's rebate of 0.40 % of the amount, in USDT, accrues on what Jeton converts (PARTNER_REBATE_RULE).
Refund	A rejected payout opens a return leg to RET_SND_A, the sender_return receiver, through the same destination-approval path; the network-fee policy absorbed applies unless a FEE_DECISION says otherwise.
Reporting	SYSTEM_SETTING reporting_currency USD; the platform pairs USDT-USD and EUR-USD convert every balance for the client-level ceiling (CLIENT.exposure_ceiling 400,000) and for the project's own cap (250,000).

Illustrative values are marked as such in Section 7.